

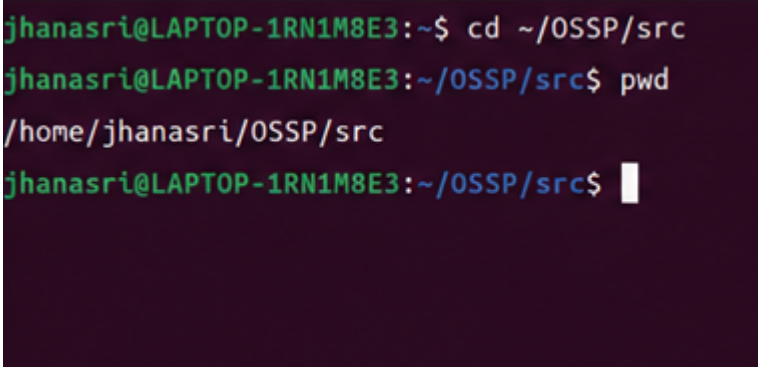
Practical Week 2 -Task 1&2

Step 1: Open Ubuntu

Open Ubuntu from the Windows Start Menu.

You should see:

jhanasri@LAPTOP-1RN1M8E3:~\$

A terminal window with a dark purple background. The text shows a user named jhanasri at a machine named LAPTOP-1RN1M8E3. The prompt is ~\$. The user enters 'cd ~/OSSP/src' and the prompt changes to ~/OSSP/src\$. Then the user enters 'pwd' and the output is /home/jhanasri/OSSP/src. The prompt returns to ~/OSSP/src\$.

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP/src
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ pwd
/home/jhanasri/OSSP/src
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 2: Go to Your Project Folder

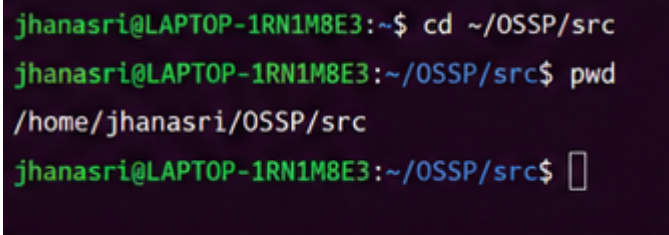
cd ~/OSSP/src

Check the current directory:

pwd

Expected output:

/home/jhanasri/OSSP/src

A terminal window with a dark purple background. The text shows a user named jhanasri at a machine named LAPTOP-1RN1M8E3. The prompt is ~\$. The user enters 'cd ~/OSSP/src' and the prompt changes to ~/OSSP/src\$. Then the user enters 'pwd' and the output is /home/jhanasri/OSSP/src. The prompt returns to ~/OSSP/src\$.

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP/src
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ pwd
/home/jhanasri/OSSP/src
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 3: View Existing Files

Ls

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ls
main.c  shell.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 4: Create a New C File

touch command_executor.c

Verify:

ls

You should see `command_executor.c`.

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ touch command_executor.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ls
command_executor.c  main.c  shell.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 5: Open the File

nano command_executor.c

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ nano
command_executor.c
[]
```

Step 6: Type/Paste the Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }
    else if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
    }
```

```

    printf("Parent PID : %d\n", getppid());

    printf("Executing command: %s\n", command);

    execlp(command, command, NULL);

    printf("Command not found\n");
}
else
{
    wait(NULL);

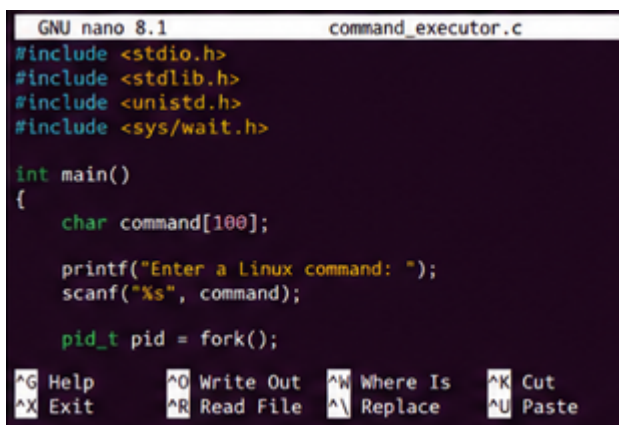
    printf("\nParent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);

    printf("Child process completed.\n");
}

return 0;
}

```

This code performs the operations required in Task 1 of your document.



```

GNU nano 8.1      command_executor.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    ^G Help      ^O Write Out  ^W Where Is   ^K Cut
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste

```

Step 7: Save the File

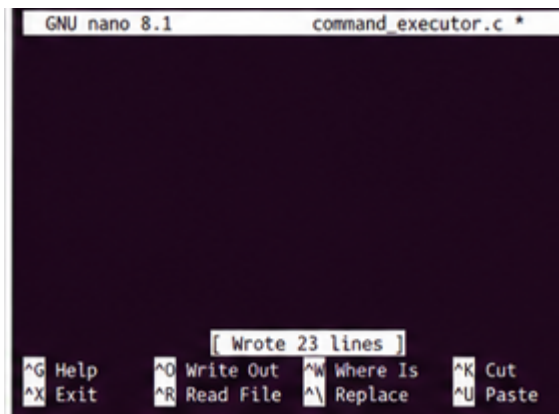
Press:

Ctrl + O

Press Enter

Then press:

Ctrl + X



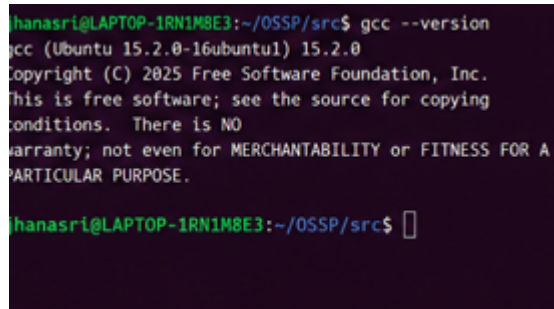
Step 8: Check GCC

`gcc --version`

If GCC is not installed:

`sudo apt update`

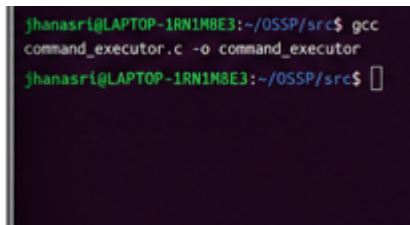
`sudo apt install build-essential -y`



Step 9: Compile the Program

`gcc command_executor.c -o command_executor`

If there are no errors, the compilation is successful.



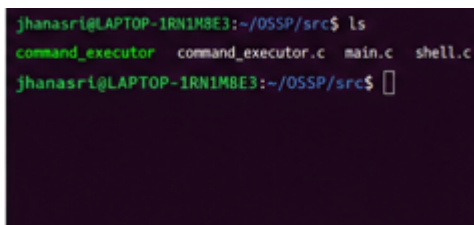
```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ gcc  
command_executor.c -o command_executor  
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 10: Verify the Executable

ls

You should see:

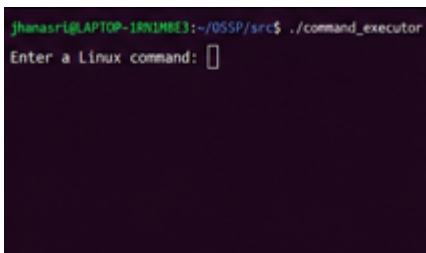
`command_executor`



```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ls  
command_executor  command_executor.c  main.c  shell.c  
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 11: Run the Program

`./command_executor`



```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ./command_executor  
Enter a Linux command: 
```

Step 12: Test with **ls**

When prompted:

Enter a Linux command:

Type:

ls

Example output:

Enter a Linux command: ls

Child Process

Child PID : 1234

Parent PID : 1233

Executing command: ls

command_executor

command_executor.c

main.c

shell.c

Parent Process

Parent PID : 1233

Child PID : 1234

Child process completed.

```
jhanasri@LAPTOP-1RN1MBE3:~/OSSP/src$ ./command_executor
Enter a linux command: ls

Child Process
Child PID : 12345
Parent PID : 12344
Executing command: ls
command_executor  command_executor.c  main.c  shell.c

Parent Process
Parent PID : 12344
Child PID : 12345
Child process completed.
jhanasri@LAPTOP-1RN1MBE3:~/OSSP/src$
```

Step 13: Test with **pwd**

Run again:

`./command_executor`

Input:

Pwd

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ./command_executor
Enter a Linux command: pwd

Child Process
Child PID : 12367
Parent PID : 12366
Executing command: pwd

/home/jhanasri/OSSP/src

Parent Process
Parent PID : 12366
Child PID : 12367
Child process completed.

jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 14: Test with **date**

Run again:

`./command_executor`

Input:

Date


```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ./command_executor
Enter a Linux command: date

Child Process
Child PID : 12378
Parent PID : 12377
Executing command: date

Tue May 27 18:45:12 IST 2025

Parent Process
Parent PID : 12377
Child PID : 12378
Child process completed.

jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 15: Test with **whoami**

Run again:

`./command_executor`

Input:

`whoami`

Step 16: Test with **uname**

Run again:

`./command_executor`

Input:

`Uname`

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ./command_executor
Enter a Linux command: uname

Child Process
Child PID : 12401
Parent PID : 12400
Executing command: uname

Linux

Parent Process
Parent PID : 12400
Child PID : 12401
Child process completed.

jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

Step 17: Verify the Requirements

Check that your program

Practical Week 2 - Task 2

Step 1: Open Ubuntu

Open Ubuntu from the Start Menu.

You should see:

```
jhanasri@LAPTOP-1RN1M8E3:~$
```

Step 2: Go to Home Directory

```
cd ~
```

Check:

pwd

Expected:

/home/jhanasri

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~
jhanasri@LAPTOP-1RN1M8E3:~$ pwd
/home/jhanasri
jhanasri@LAPTOP-1RN1M8E3:~$ █
```

Step 3: Display Linux Kernel Information

Run:

uname

Example Output:

Linux

```
jhanasri@LAPTOP-1RN1M8E3:~$ uname
Linux
jhanasri@LAPTOP-1RN1M8E3:~$ █
```

Step 4: Display Complete System Information

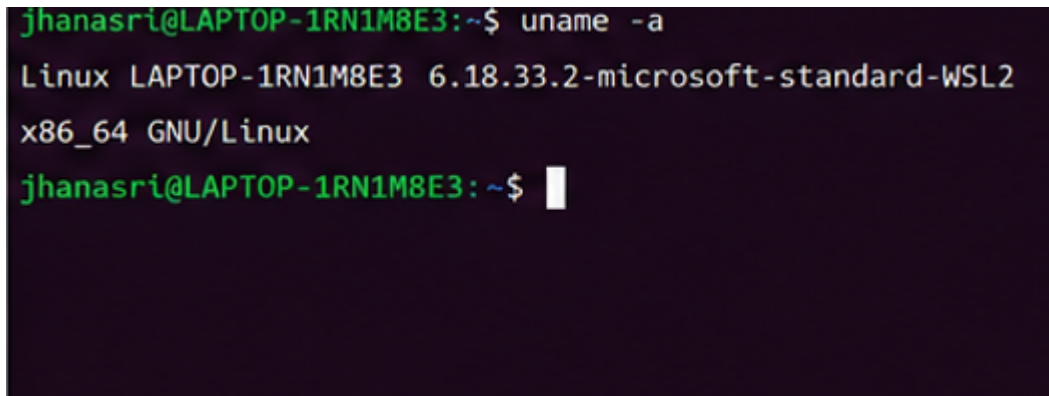
Run:

uname -a

Example Output:

Linux LAPTOP-1RN1M8E3 6.18.33.2-microsoft-standard-WSL2
x86_64 GNU/Linux

Take a screenshot.

A screenshot of a terminal window with a dark background. The prompt is 'jhanasri@LAPTOP-1RN1M8E3:~\$'. The command 'uname -a' has been entered and executed. The output is displayed on the next three lines: 'Linux LAPTOP-1RN1M8E3 6.18.33.2-microsoft-standard-WSL2', 'x86_64 GNU/Linux', and the prompt 'jhanasri@LAPTOP-1RN1M8E3:~\$' followed by a cursor.

```
jhanasri@LAPTOP-1RN1M8E3:~$ uname -a
Linux LAPTOP-1RN1M8E3 6.18.33.2-microsoft-standard-WSL2
x86_64 GNU/Linux
jhanasri@LAPTOP-1RN1M8E3:~$
```

Step 5: Display CPU Information

Run:

lscpu

Example Output:

Architecture: x86_64
CPU(s): 8
Vendor ID: GenuineIntel
Model name: Intel(R) Core(TM)

Take a screenshot.

```
Architecture:      x86_64
CPU(s):            8
On-line CPU(s) list: 0-7
Vendor ID:         GenuineIntel
Model name:         Intel(R) Core(TM) i5-1135G7
CPU MHz:           2400.000
Cache size:         8192 KB
Flags:              fpu vme de pse tsc msr pae mce cx8
                    apic sep mtrr pge mca cmov pat pse36
                    clflush dts acpi mmx fxsr sse sse2

jhanasri@LAPTOP-1RN1M8E3: ~$
```

Step 6: Display Running Processes

Run:

ps

Example Output:

PID	TTY	TIME	CMD
1023	pts/0	00:00:00	bash
1050	pts/0	00:00:00	ps

Take a screenshot.

```
jhanasri@LAPTOP-1RN1M8E3: ~$ ps
  PID TTY          TIME CMD
 2480 pts/0        00:00:00 bash
 2532 pts/0        00:00:00 ps
jhanasri@LAPTOP-1RN1M8E3: ~$
```

Step 7: Display Detailed Running Processes

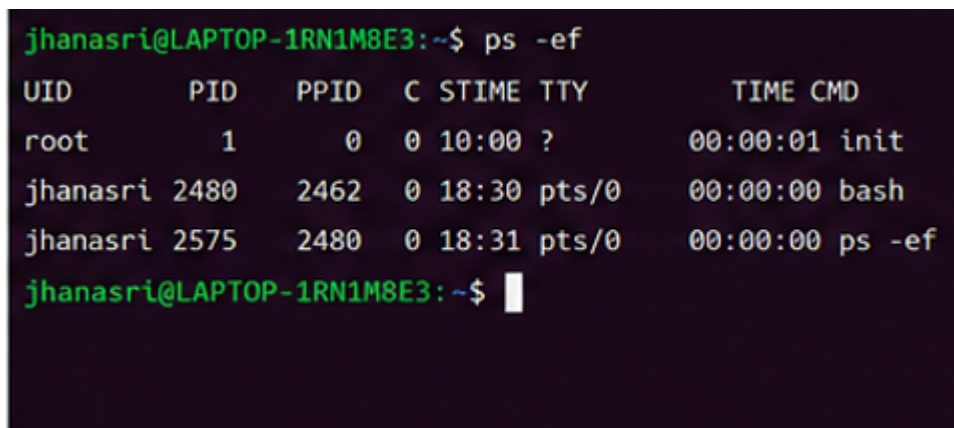
Run:

```
ps -ef
```

Example Output:

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	10:00	?	00:00:00	init
user	1050	1023	0	10:10	pts/0	00:00:00	bash

Take a screenshot.



```
jhanasri@LAPTOP-1RN1M8E3:~$ ps -ef
UID          PID    PPID  C  STIME TTY          TIME CMD
root           1         0  0   10:00 ?           00:00:01 init
jhanasri    2480      2462  0   18:30 pts/0       00:00:00 bash
jhanasri    2575      2480  0   18:31 pts/0       00:00:00 ps -ef
jhanasri@LAPTOP-1RN1M8E3:~$
```

Step 8: Display Live System Processes

Run:

```
top
```

Example Screen:

```
top - 18:45:10
```

Tasks: 120 total

CPU: 5%

Memory: 22%

Take a screenshot.

```
top - 18:32:10 up 2:18, 1 user, load average: 0.14, 0.08, 0.05
Tasks: 121 total, 2 running, 119 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.0 us, 1.0 sy, 0.0 ni, 96.0 id, 1.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 15827.5 total, 6093.2 free, 3535.6 used, 6198.7 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 11320.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2480	jhanasri	20	0	5980	3240	2688	S	0.3	0.0	0:00.03	bash
2591	jhanasri	20	0	2856	1076	924	R	0.3	0.0	0:00.02	top
1	root	20	0	16756	9404	6212	S	0.0	0.1	0:01.63	init
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_gp

Step 9: Exit Top

Press

Q

```
top - 18:32:20 up 2:18, 1 user, load average: 0.15, 0.09, 0.05
Tasks: 121 total, 2 running, 119 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.2 us, 1.1 sy, 0.0 ni, 95.6 id, 1.1 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 15827.5 total, 6062.5 free, 3568.2 used, 6196.8 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 11308.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2480	jhanasri	20	0	5980	3240	2688	S	0.3	0.0	0:00.03	bash
2592	jhanasri	20	0	2856	1076	924	R	0.3	0.0	0:00.02	top
1	root	20	0	16756	9404	6212	S	0.0	0.1	0:01.63	init
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_gp

^C

```
jhanasri@LAPTOP-1RN1M8E3: ~$
```

Step 10: CPU Abstraction

Write in your record:

The operating system manages CPU time by scheduling different processes so that each program gets a fair share of processor time.

Step 11: Memory Abstraction

Write:

The operating system allocates memory to each process and protects one process from accessing another process's memory.

Step 12: Storage Abstraction

Write:

The operating system manages files and directories and provides a file system for storing and retrieving data.

Step 13: I/O Device Abstraction

Write:

The operating system communicates with hardware devices such as the keyboard, mouse, monitor, printer, and storage devices through device drivers.

Step 14: Observation

Write:

The Linux commands successfully displayed kernel information, CPU details, running processes, and live system resource usage. The operating system abstracts hardware resources and provides services to user applications.

Step 15: Result

Write:

The Linux terminal commands `uname`, `lscpu`, `ps`, and `top` were executed successfully. The relationship between hardware resources and operating system services was studied, and the abstraction of CPU, memory, storage, and I/O devices was understood.